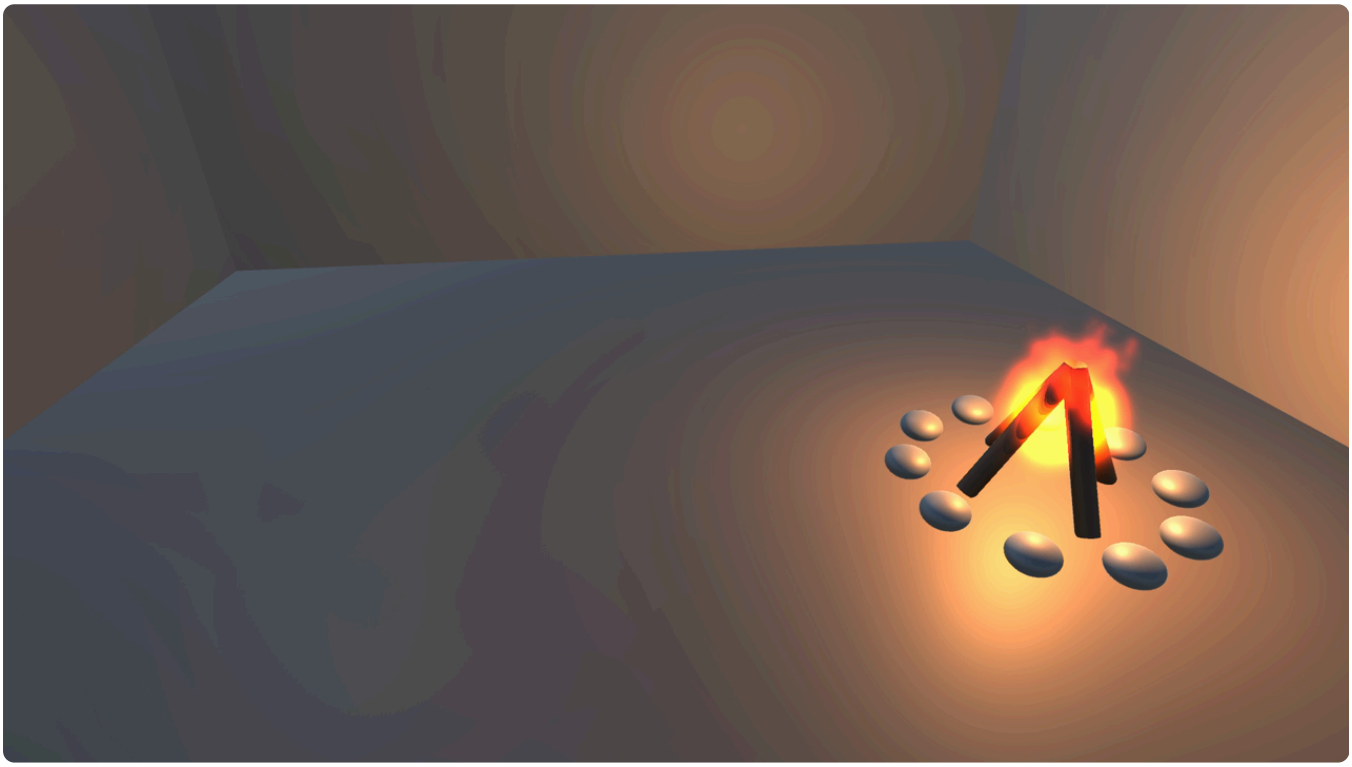
COMSM0127 Immersive Interaction and Audio Design

Week 3, Lab B: Sound Design and Dynamic Audio

Dr Richard Hemming, Chris Bevan and Pete Bennett



THIS WORKSHEET

This is the **Worksheet**: every step, with pictures. The **Quick Card** is a two-page summary of the same steps, for your desk. If they ever seem to disagree, follow this Worksheet.

Words in **bold** are the key words. A short list of key words is at the end.

Today's lab

Last week you put a spatialised sound in a room. Today you **make** a sound, and make it **change**.

At the start you heard a **mystery sound**. You will rebuild it in two ways:

- **Part A, Recorded:** record its parts, edit them, and play them back so it **never repeats**.
- **Part B, Procedural:** make its parts from **code**, and make them **respond** to the game.

Listen to the mystery sound again at any time: in Unity's Project window, open `Assets/W3B_Fire/Audio/Mystery` and click **mystery_fire**. Press the play button at the bottom of the Inspector.

	First half	Second half
Groups 1, 2	**Part A: Recorded**	Part B: Procedural
Groups 3, 4	**Part B: Procedural**	Part A: Recorded

I will call the swap. When I call everyone back, each group plays its fire to the room.

By the end of the lab you will have

1. Listened to the mystery sound and named the parts it is made of.
2. **Part A:** recorded those parts and played them back so no two plays are the same.
3. **Part B:** made those parts from code and made one respond to the fire's **Intensity**.
4. Changed **one line of C#** in each part.
5. Played your fire to the room.

Basic goal: Campfire A and Campfire B both working. **Stretch goals:** at the end of this worksheet.

LEARNING OUTCOMES

Today builds towards the unit's learning outcomes: **LO1** design and build with spatialised audio (both fires); **LO2** evaluate (recorded or procedural: which works better?); **LO3** ethics and safety (next page); **LO4** the technologies (Unity, Steam Audio, Audacity, C#); **LO5** communicate (playing your fire to the room); **LO6** collaboration (your group). In your coursework, this is **"Audio techniques demonstrated"**.

Ethics and safety: read this first

Learning outcome 3: the ethical and safety aspects of interaction and spatialised audio.

RECORDING

1. **Ask first.** Never record a person without asking them. If someone's voice is caught on a take, **delete that take**.
2. **Stay in the room.** Record in a quiet **corner of the lab**. One group per corner.
3. **Never leave the recorder alone.** Someone in your group holds it at all times.
4. **Format the SD card before you hand the recorder back.** Once your files are copied off, delete everything on the card (on the Zoom: **Menu > SD Card > Format**). Recordings left on the card break the University's privacy rules.

YOUR EARS

1. Set the computer volume **low** before you press Play. Then turn it up slowly.
2. **Code can make sound suddenly loud.** In Part B, start each layer's **Gain** low (about 0.1) and raise it slowly.
3. After you change any code, take your headphones **off your ears** for the first Play.
4. Too loud? Click **Mute Audio** at the top of the Game view, or press **Stop**.
5. Never type a big number into **Gain** or **Bursts Per Second** with headphones on.

Get the project

Use **GitHub Desktop** (free). Do **not** use Download ZIP: Windows blocks the files inside it, and macOS blocks the Steam Audio plugin. GitHub Desktop avoids both.

1. Go to **github.com/RichSF1210/W3B_FireLab**. Click the green **Code** button, then **Open with GitHub Desktop**. (In GitHub Desktop you can also use **File > Clone Repository > URL** and type `RichSF1210/W3B_FireLab`.)
2. **Local Path:** choose a folder **on this computer** that does **not** sync to OneDrive or iCloud. Windows: `C:\Unity\`. Mac: a folder in your home folder. Click **Clone**. You do not need a GitHub account.
3. **Your own computer?** Open **Unity Hub** once and **sign in**. If it asks for a licence, add the free **Personal** licence (Unity Hub > Settings > Licenses > Add).
4. **Open the project in Unity (6000.3.19f1):**
   - **Windows:** in File Explorer, open `Assets/W3B_Fire/Scenes` and double-click **W3B_Student_Shell**.
   - **Mac:** in Unity Hub, click **Add > Add project from disk**, choose the **W3B_FireLab** folder, and open it.

   The first open takes a few minutes.
5. In Unity's **Project** window, open `Assets/W3B_Fire/Scenes` and double-click **W3B_Student_Shell**. **This is the scene you work in.** Press Play. Walk with **W A S D**; hold the right mouse button to look.

> IMPORTANT
> **Never press Commit or Push** in GitHub Desktop. Your copy is yours to change. You cannot change the original.

In the project	What it is
`W3B_Student_Shell`	**You start here.** A room and two silent campfires: **Campfire A (Recorded)** for Part A and **Campfire B (Procedural)** for Part B. Each has three empty layers, **Layer 1, 2, 3**: one part of the sound per layer.
`W3B_Complete_Reference`	Both campfires finished. Look at it **after** you try, to compare.
`W3B_Demo`, `W3B_Finale`	The lecturer's demos. Take them away and explore them later.
`Audio/My_Recordings`	The folder for **your** recordings.
`Audio/Fire_Recorded`	Example recordings, if yours do not work.

Save the scene often: Ctrl+S (Windows) or Cmd+S (Mac).

First: listen

Headphones on, eyes closed. Play the mystery sound again (`Assets/W3B_Fire/Audio/Mystery`). As a group, agree:

- How many **separate parts** can you hear?
- Describe each part: **high or low**? **Constant** (goes on steadily) or **short** (separate bursts)?
- Which part tells you most strongly **what the sound is**?

Give each part a **name of your own**. Fill in this table. You will use it in both parts.

Part (your name for it)	Constant or short?	Part A: files you made	Part B: building block

RULE

Constant part = one **loop** file. **Short part** = **several** short files (at least four). One part per layer.

Work on one campfire at a time

Both campfires are in the same room. So you hear only the one you are working on:

1. **Stop** Play mode first (changes made while playing are lost).
2. In the **Hierarchy**, click the campfire you are **not** working on.
3. At the top of the **Inspector**, **untick** the box next to its name. It goes grey and silent.
4. Click your campfire and carry on. **Tick both again** when everyone plays their fires.



Hear one layer at a time

Select a layer. In its **Audio Source**, tick **Mute** to silence it; untick to hear it again. This works while playing.

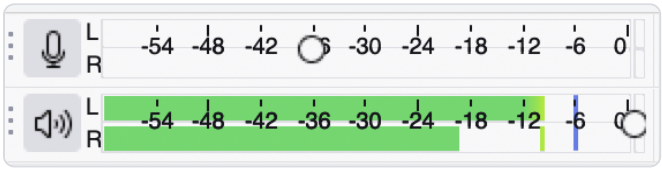
A

Part A: Recorded

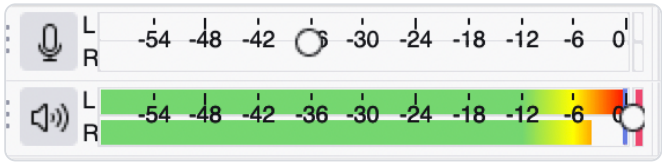
A1. Record

Take a **Zoom recorder** to a quiet **corner of the lab**. For each part on your list, find something that makes a **similar** sound: scrunch, snap, spray, blow, rub, tear.

- **Get close** to the sound. Keep it quiet around you.
- **Set the level:** the loudest moments should reach about **-12 dB** on the Zoom's meter, and **never the red**. Red means **clipping**: the sound is damaged and **cannot be fixed later**. If a take clips, record it again, a little quieter.
- Record **several takes** of each part. About 10 seconds each is plenty. For a short part, record **at least four** different sounds.
- Say the part's name **before** each take, so you can find it later.



Good: the meter stays out of the red.



Bad: the meter hits the red. Record again.

A2. Get the files off

1. Take your Zoom (or its SD card) to the **shared card reader**. Take turns.
2. Copy your WAV files into a folder on your computer, for example **Desktop/MyRecordings**. (If the reader is on another computer, move the files with a USB stick or OneDrive.)
3. When everything is copied, **format the SD card** (see Ethics and safety).

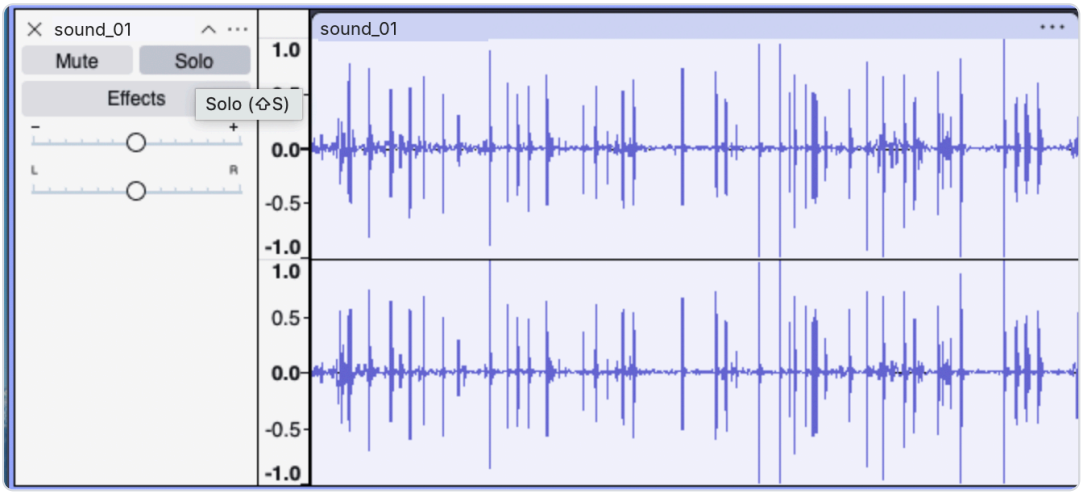
WAITING?
Start the next steps with the example files in `Assets/W3B_Fire/Audio/Fire_Recorded`, and swap in yours later.

A3. Edit in Audacity

The plan: **one file = one sound**. You will make **one loop** for each constant part and **several short files** for each short part.

Get ready

1. **Listen** to each file and **rename** it after what it sounds like, with a number: `name_01`, `name_02`, `name_03`. Use names you will recognise later.
2. Open **Audacity**. Select **all** your files and **drag them in together**. Each file becomes a **track**.
3. Press **Solo** on a track to hear **only** that sound.

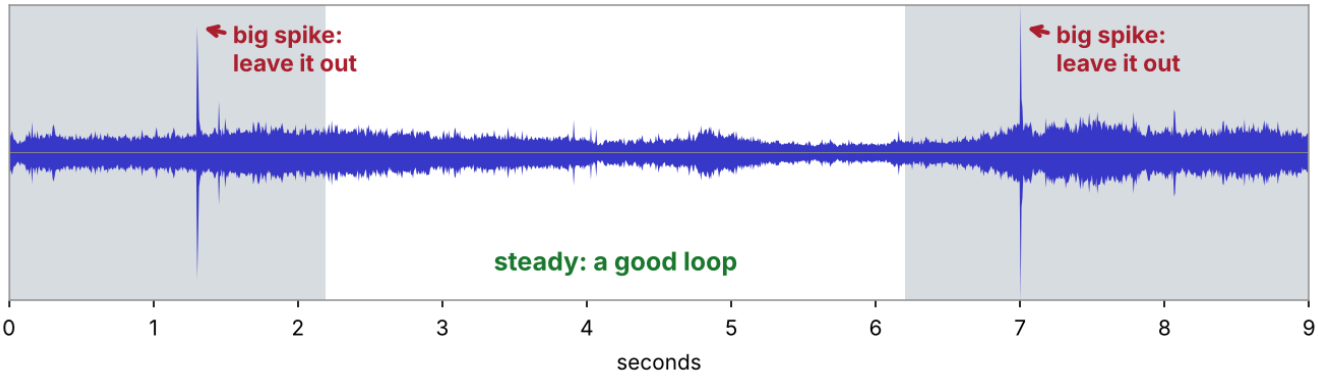


Solo: hear one track on its own.

To do this	Do this
Play / stop	**Space** (plays just the selection, if there is one)
Select	Click and drag across the waveform
Zoom in on the selection / see it all	**Ctrl+E** / **Ctrl+F** (Mac: Cmd)
Undo	**Ctrl+Z** (Mac: Cmd+Z), as often as you like

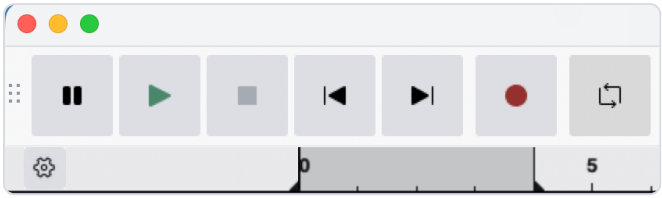
Constant parts: make one loop

A loop: choose 3 to 5 seconds that stay steady. Leave out the big spikes.

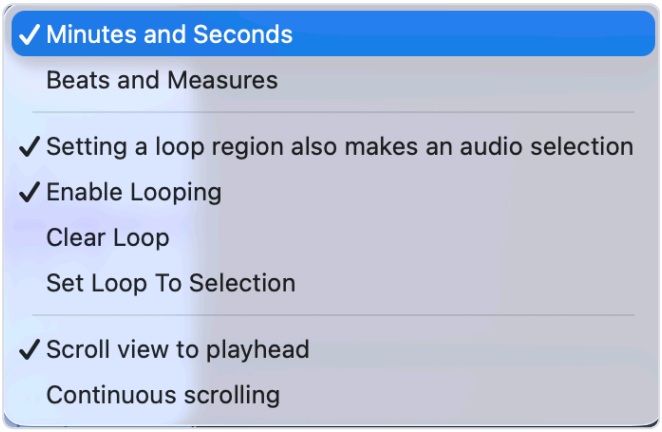


1. Click the **loop button** to turn looping on.

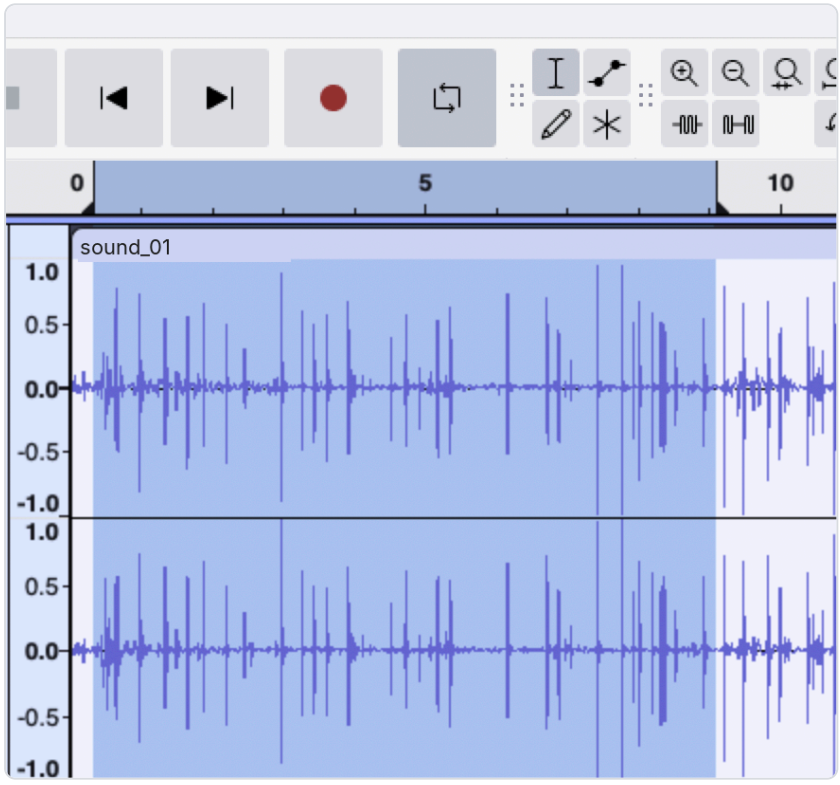
2. Drag across the **timeline ruler** (the numbers at the top) over **3 to 5 steady seconds**. Press **Space**: it plays round and round. Move the edges until you **cannot hear the join**.
3. Right-click the ruler and tick **Setting a loop region also makes an audio selection**.
4. **Check the audio is highlighted** on the track itself: **blue over the waveform**. Only highlighted audio is exported. If there is no blue, drag across the same part of the waveform.



The loop button (lit = on).

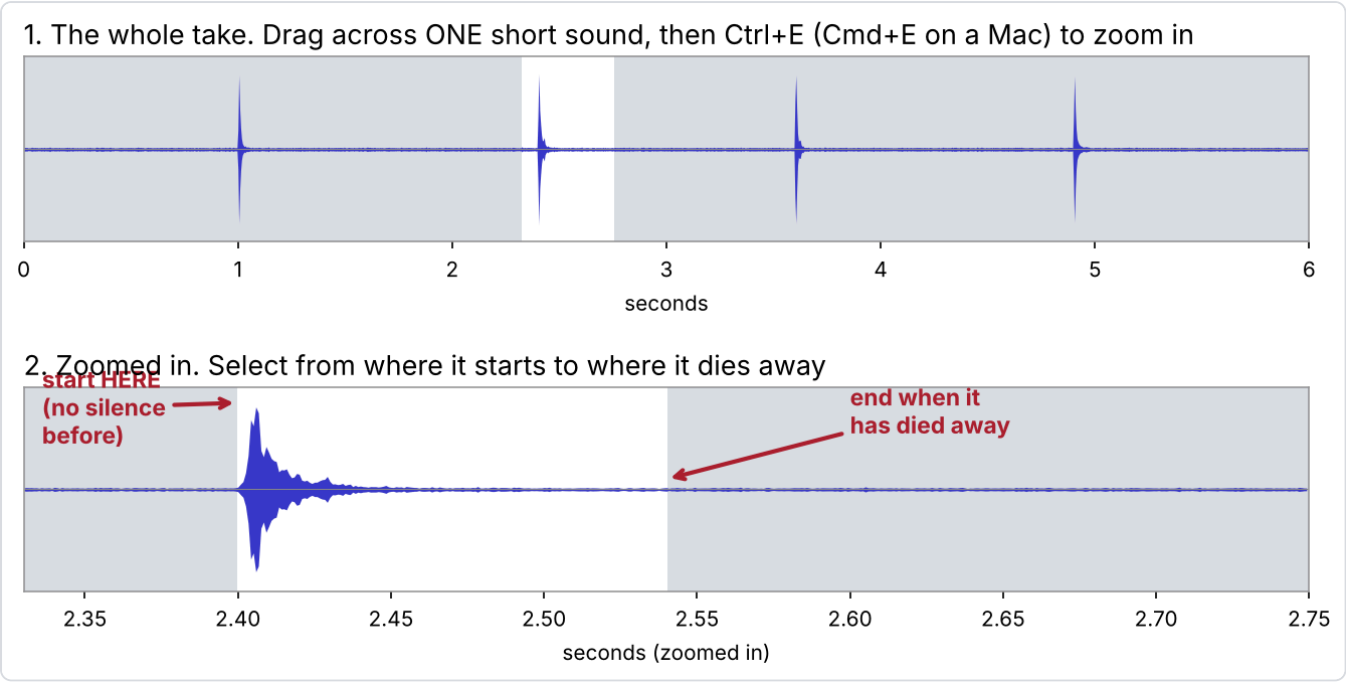


Right-click the ruler.



Blue over the waveform = what will be exported.

Short parts: make several files

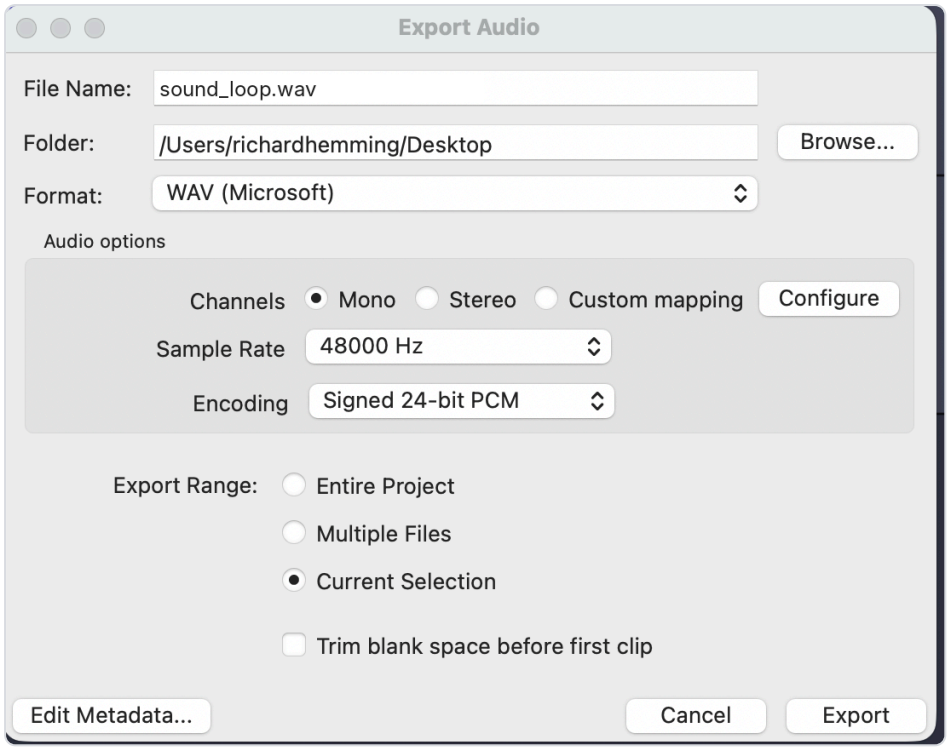


1. Drag across **one** short sound. Press **Ctrl+E** to zoom in.
2. Select from **its very start** to where it has **died away**. No silence before it, or it will play late. Press Space to check.
3. Export it (below) as `name_01`. Then select the **next** sound and export it as `name_02`. Make **at least four**.

Export (for every file)

File > Export Audio, then set:

Setting	Choose
File Name	`name_loop` for a loop; `name_01`, `name_02` ... for short sounds. Lower case, no spaces.
Folder	Your Desktop recordings folder
Format	**WAV**
Channels	**Mono** (see below)
Sample Rate	**48000 Hz**
Encoding	**Signed 24-bit PCM**
Export Range	**Current Selection** (important!)



WHY MONO?

The Zooms record in **stereo** (left and right). But your fire is **one point** in 3D space. **Steam Audio** places a **mono** sound cleanly as you walk round it. A stereo file has its own left and right, which fights that. So export **Mono**.

CHECK

Do you have **at least one loop** AND **a set of at least four short files**? Then go to Unity.

A4. Into Unity

1. In Unity's **Project** window, open `Assets/W3B_Fire/Audio/My_Recordings`.
2. **Drag your WAV files** from your Desktop folder into it.
3. **Stereo file?** Click it, tick **Force To Mono** in the Inspector, and click **Apply**. (Not needed if you exported Mono.)
4. Click a file: the Inspector shows its waveform. Press play at the bottom to check it.

You may see `.meta` files in File Explorer or Finder. Unity makes them itself: **ignore them**, and never drag them.

NO RECORDINGS, OR NOT HAPPY WITH THEM?

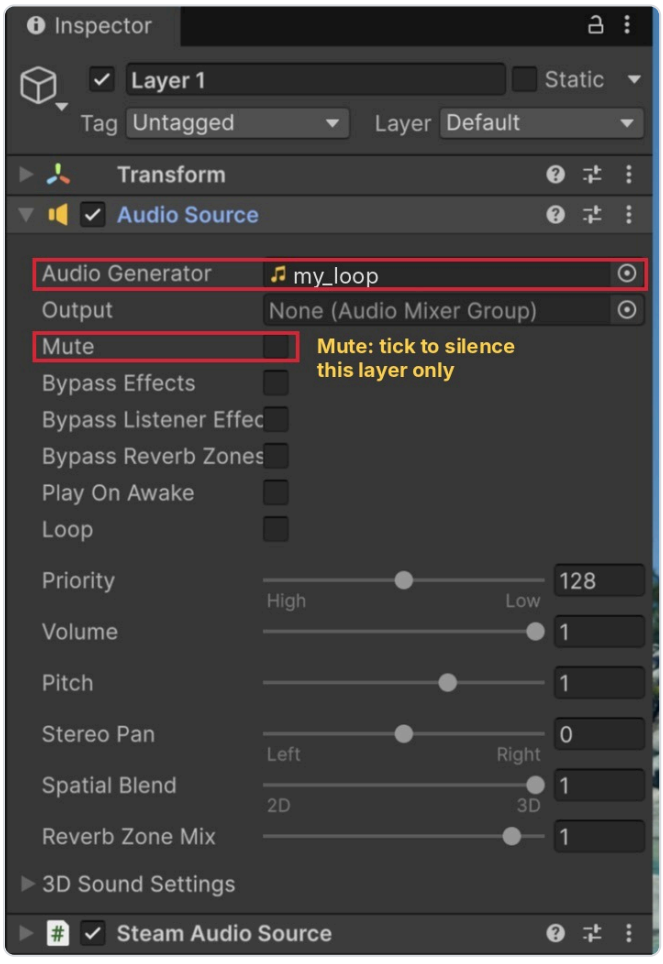
Use ours. They are already in `Assets/W3B_Fire/Audio/Fire_Recorded`: **loops** (files ending `_loop`) and **short sounds** (files numbered `_01` to `_08`). You can mix yours with ours.

A5. Play them back

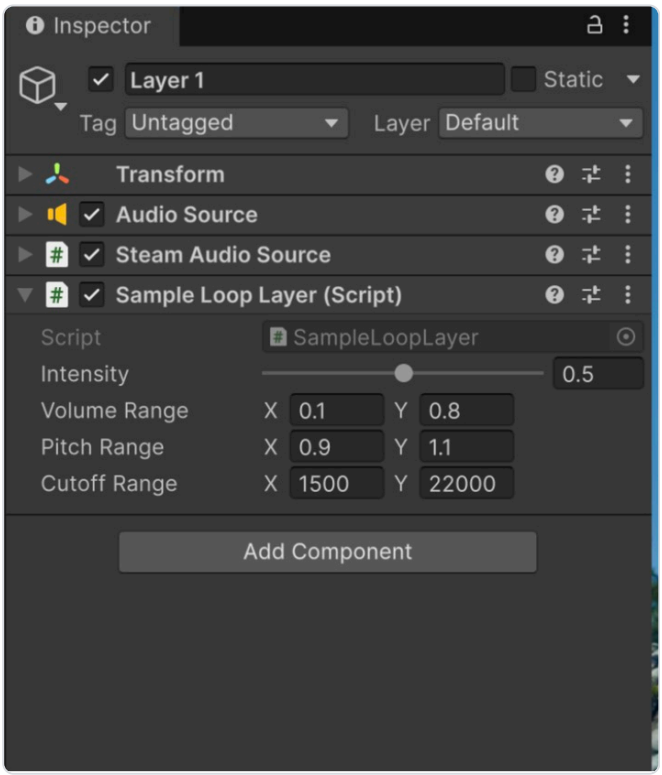
Work on **Campfire A (Recorded)**. One part per layer.

Constant part (a loop)

1. Select a layer, for example **Layer 1**. Click **Add Component**, type **SampleLoopLayer**, press Return.
2. Drag your loop onto the **Audio Generator** field of the layer's **Audio Source**.
3. Press Play. The **Console** must not say "no clip assigned".



Audio Source: your loop goes in Audio Generator.



A finished loop layer.

Short part (several files)

1. Select another layer, for example **Layer 2**. **Add Component > MyRandomPlayer**.
2. Click the **padlock** at the top right of the Inspector. It now stays on this layer.
3. In the Project window, select **all** your short files: click the first, then **Shift-click** the last.
4. Drag them onto the word **Clips** (the heading, not a row) and let go. The list fills, one row per file.
5. Click the padlock again to unlock. **Check:** Clips shows **at least 4**.



SampleLoopLayer (a constant part) and MyRandomPlayer (a short part).

Press Play and listen. Then listen again. **What gives it away as a recording?**

A6. Your one line: make it vary

CODE

1. The script is `Assets/W3B_Fire/Scripts/Student/MyRandomPlayer.cs`. Open it: double-click the grey **Script** field in the My Random Player component, or double-click the file in the Project window.
2. It opens in your **code editor**. Search for **pitch** (Ctrl+F or Cmd+F). It is near line 60.
3. **Predict first:** read the comment above the line. What do you expect to hear? Write it down.
4. **Delete the two slashes** `//` at the start of this line:

```
// TODO: YOUR ONE LINE. Delete the two slashes at the start of the next line.
source.pitch = Random.Range(0.85f, 1.2f);
//   A random pitch every play: 1.0 is normal, 0.85 lower, 1.2 higher.
```

5. **Save** (Ctrl+S or Cmd+S). Go back to Unity and **wait for it to compile the C# code** (the spinner at the bottom right). Then press Play. Were you right?

On a Mac, if the script opens in TextEdit: **Unity > Settings > External Tools > External Script Editor**, choose Visual Studio Code.

The files never change. The playback never repeats. That is dynamic audio from recordings. But a recording's **shape** is fixed: code can only change **how** it is played (which file, when, how loud, what pitch). That is the limit of Part A, and the reason for Part B.

B

Part B: Procedural

YOUR EARS

Code can make sound **suddenly loud**. Computer volume **low**. Each layer's **Gain** starts at about **0.1**. Headphones **off** for the first Play after a code change.

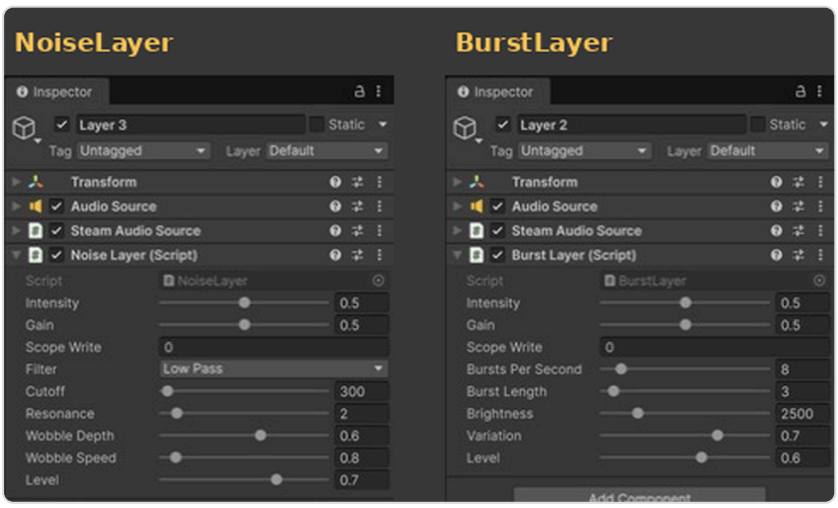
B1. Two building blocks

No audio files. These scripts make sound from numbers, as it plays. Making or changing sound with maths, one number at a time, is called **DSP** (digital signal processing). Every procedural sound gets its shape from four steps:

Step	What it decides	NoiseLayer (constant parts)	BurstLayer (short parts)
Source	The raw material	White noise	White noise
Filter	What it seems made of	Cutoff: low = roar, high = hiss	Brightness: high = small, hard things
Envelope	How it moves in time	A slow wobble: it breathes	Instant start, quick fade: a snap
Randomness	That it never repeats	Random wobble	Random timing and size

Work on **Campfire B (Procedural)**. **One script per layer**: each script takes over its layer's Audio Source, so two on one layer means only one is heard.

1. In the Hierarchy, open **Campfire B** and select **Layer 1**. **Add Component**, type **NoiseLayer**. Use it for a **constant** part.
2. Select **Layer 2**. **Add Component**, type **BurstLayer**. Use it for a **short** part.
3. **Layer 3:** another NoiseLayer or BurstLayer for your third part.



NoiseLayer and BurstLayer, each on its own layer.

B2. Sculpt by ear, then read the code

One layer at a time. Mute the others (Audio Source > Mute).

1. Add **NoiseLayer**. Press Play. Move its sliders **while you listen**. **Describe what each slider does** to the sound, in your own words.
2. **Write your best values down.** Slider changes made while playing **reset when you stop**. Stop, then type the values back in.
3. Now **open the script** (`Assets/W3B_Fire/Scripts/Student/NoiseLayer.cs`) and read the comment above each setting. **Was your ear right?**
4. Do the same for **BurstLayer** on Layer 2.
5. Unmute both layers and listen **together**. Balance their **Level**s.

- NoiseLayer, Low Pass, low cutoff: what does it remind you of? High Pass, high cutoff?
- BurstLayer, fast, short and bright: what is it? Slow, long and dark?

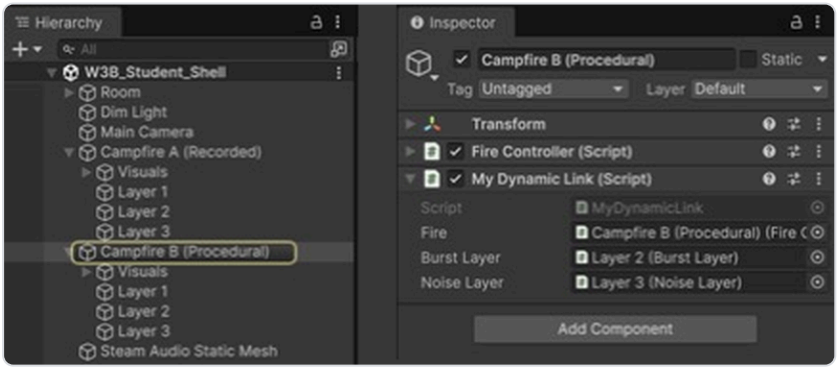
RESONANCE DOES NOTHING?

Resonance only works when the filter is **Band Pass**. Switch to Band Pass and sweep it: the noise narrows into a whistle.

B3. Your one line: make it respond

Campfire B has a **FireController** with an **Intensity** slider: 0 is embers, 1 is a blaze.

1. Select **Campfire B**. **Add Component > MyDynamicLink**.
2. **Before you press Play**, fill the slots by dragging from the Hierarchy: **Fire** = Campfire B; **Burst Layer** = the layer with your BurstLayer; **Noise Layer** = the layer with your NoiseLayer. (Fill them on Campfire B in the Hierarchy, not on the script file.)
3. Open the script: `Assets/W3B_Fire/Scripts/Student/MyDynamicLink.cs`. Find **TODO**.



MyDynamicLink on Campfire B, with all three slots filled.

CODE

Pick ONE line to start, A or B. Delete the two slashes `//` at the start of **that line only**:

```
// A) MORE SNAPS as the fire grows: 1 burst a second at embers, 40 at a blaze.
// if (burstLayer) burstLayer.burstsPerSecond = Mathf.Lerp(1f, 40f, i);

// B) A BRIGHTER FIRE as it grows: the filter opens from 200 Hz to 2,000 Hz.
// if (noiseLayer) noiseLayer.cutoff = Mathf.Lerp(200f, 2000f, i);
```

1. **Save**, go back to Unity and **wait for it to compile the C# code**. Press Play.
2. Select Campfire B and drag **Intensity** from 0 to 1. **Is it more than volume?**
3. Then put the slashes back and try the **other** line. **Stretch:** both lines at once.

Why one at a time? So you can hear exactly what each line does. The line runs about **60 times a second** (inside `Update()`), so the sound follows the slider all the time.

`Mathf.Lerp(1, 40, i)` goes from 1 to 40 as `i`, the fire's Intensity, goes from 0 to 1. This is **RTPC**, **Real-Time Parameter Control**: a value from the game drives a parameter of the sound. Game audio tools such as Wwise and FMOD are built on this idea.

A yellow message in the Console such as "the Noise Layer slot is empty" means a slot is not filled. Stop, fill it, and press Play again.

B4. Read the code

Answer these by **reading the scripts**, not by guessing. Write your answers here.

Question	Your answer
In **NoiseLayer.cs**, which line makes the level wobble, and what does it use?	
In **BurstLayer.cs**, what decides how fast a burst dies away?	
What would `Mathf.Lerp(40f, 1f, i)` do to the fire?	
Change one number in your line (for example `Mathf.Lerp(2000f, 200f, i)`: a fire that gets **darker** as it grows). What did you hear? Does it feel right?	

Stuck?

Problem	Try this
No sound	Is the campfire ticked? Is the layer muted? Is the Audio Source volume up? In Part B, is Gain above 0?
Hearing both campfires	Untick the campfire you are not working on (see "Work on one campfire at a time").
Red error in the Console	A typo in the line you changed. Double-click the error: it opens the line. Check it ends with `;` and the spelling matches exactly.
NullReferenceException	An empty slot. Stop, fill the slot, and click **Clear** in the Console.
An old error will not go away	Click **Clear** in the Console. The status bar shows the last error until you clear it.
My changes vanished	You made them while playing. Make them again when stopped.
"no clip assigned"	Drag your loop onto the Audio Source's **Audio Generator** field.
Clips will not fill	Lock the Inspector first (padlock), then drag onto the word **Clips**.
Only one of two scripts plays	Two procedural scripts on one layer. Put each on its own layer.
Resonance does nothing	Set the filter to **Band Pass**.
Recorder trouble, or out of time	Use `Assets/W3B_Fire/Audio/Fire_Recorded`.
"No valid Unity Editor license found"	Open Unity Hub, sign in, add the free Personal licence.
Mac: "audioplugin_phonon.bundle is damaged"	Press **Cancel**, never Move to Bin. You downloaded the ZIP: use GitHub Desktop instead.
Too loud	Stop. Computer volume down. Gain down. Mute Audio in the Game view.

Ask us. That is what we are here for.

Stretch

Each card gives you a **goal**, **where to start**, **what success sounds like** and a question to **push further**. Hints, not answers.

Card	Goal and where to start	Success sounds like	Push further
1 Combine	One fire with a recorded layer AND a procedural layer. Start: Campfire A, add a NoiseLayer to its Layer 3.	The two blend into one fire.	Which parts work better recorded, and which from code? Why?
2 Compare	Open `W3B_Complete_Reference`. Walk between its two campfires.	You can hear what each fire does differently.	What would you change in yours?
3 Interact	Add `AddLogOnKey` to a campfire, drag the campfire into its Fire slot, press **L** to throw on a log.	The fire flares, then settles.	In your exhibit, a VR hand gesture or grab could call `AddLog()` instead.
4 A fire that changes by itself	Intensity is just a number from 0 to 1. In MyDynamicLink, make your own `i` instead of `fire.CurrentIntensity`: from time (`Time.time`), from how close the player is, or from a slow wave. Keep it between 0 and 1 with `Mathf.Clamp01`.	The fire grows or dies with no slider.	Which source of change feels most like a real fire? Then compare with the **Burn Down** box on FireController.
5 Your own building block	Write the sound for Layer 3 yourself. Start: `Scripts/Layers/OldExercises` (read its README), or copy NoiseLayer.cs and change `NextSample()`.	A part of the fire that is not just filtered noise.	Can it respond to `intensity`?
6 A new room	Put your fire in a different space. As in Week 2 Lab B, give a room's surfaces **Steam Audio materials** (Brick, Carpet, Concrete... in `Plugins/SteamAudio/Resources/Materials`) and move the campfire inside.	The same fire sounds different in a different room.	Which material makes it feel most like a real fireplace, and why?

When I call everyone back

Bring **one fire** to play through the speaker. Be ready to tell us:

1. What parts did you hear in the mystery sound? What did you call them?
2. Which part worked better **recorded**, and which **procedural**?
3. What did your lines of code change? **Which number did you change, and what did it do?**

Before you leave: **format the SD card** and hand the recorder back.

FOR YOUR COURSEWORK

On Monday we work on the audio in your projects. **If you can**, bring one sound from your exhibits that **could** be dynamic: one that changes when the player does something, or as the world changes. We can try building it together.

Key words

Dynamic audio	Any sound that changes during play (Collins, 2008).
Interactive	Changes because the player did something (a log thrown on).
Adaptive	Changes because the state of the world changed (the fire burns down).
Procedural	Made while the game runs, by code following rules. No audio file.
Constant part / short part	A part that goes on steadily (one loop) / a part made of separate bursts (several files).
Loop	A recording that plays round and round with no audible join.
Clipping	A recording too loud for the recorder: the meter hits the red, and the sound is damaged.
Mono / stereo	One channel / two channels (left and right). Use mono for a sound placed in 3D.
Sample	One number in a digital sound. About 48,000 a second.
DSP	Digital signal processing: making or changing sound with maths, one sample at a time. A filter is DSP.
Source, filter, envelope	The raw material (noise) / what it seems made of / how it moves in time.
Cutoff, Hz	Where a filter cuts, in hertz (cycles a second). Low = rumble, high = hiss.
RTPC	Real-Time Parameter Control: a value from the game drives a parameter of the sound.
Lerp	`Mathf.Lerp(a, b, i)` : go from a to b as i goes from 0 to 1.
Compile	Unity turns your C# into a program it can run. Wait for it after every save.

References

Collins, K. (2008). *Game sound: An introduction to the history, theory, and practice of video game music and sound design*. MIT Press.

Farnell, A. (2010). *Designing sound*. MIT Press.

Sentance, S., Waite, J., & Kallia, M. (2019). Teaching computer programming with PRIMM: A sociocultural perspective. *Computer Science Education, 29*(2–3), 136–176. https://doi.org/10.1080/08993408.2019.1608781

Steam Audio 4.7.0, Valve Corporation (Apache License 2.0). Unity 6000.3.19f1, Unity Technologies. Audacity, Audacity Team. Recordings in the project by Richard Hemming.